Hospital AI Agent

Technology Stack and Development Environment Plan

---

1. Introduction

The Hospital AI Agent is a full-stack AI-powered web application.

The system requires several technologies to build the frontend, backend, database, authentication system, AI Agent, and notification services.

This document defines the proposed technology stack and development environment for the project.

---

2. Technology Stack Overview

The proposed technology stack is:

Layer	Technology
Frontend	React.js
Backend	Python + FastAPI
Database	PostgreSQL
AI Agent	Python-based AI Agent
Authentication	JWT
API Architecture	REST API
ORM	SQLAlchemy
Frontend Styling	CSS / Tailwind CSS
Data Validation	Pydantic
Version Control	Git
Repository Hosting	GitHub

---

3. Frontend Technology

React.js

React.js will be used to build the user interface of the Hospital AI Agent system.

The frontend will provide different interfaces depending on the user's role.

Examples include:

- Administrator Dashboard.
- Doctor Dashboard.
- Patient Dashboard.
- Appointment Pages.
- Doctor Management Pages.
- Patient Management Pages.
- AI Agent Chat Interface.

Main Frontend Pages

The initial system may include:

Authentication

- Login Page.
- Registration Page.

Administrator

- Admin Dashboard.
- Doctor Management.
- Patient Management.
- Appointment Management.
- Reports.

Doctor

- Doctor Dashboard.

- Doctor Schedule.
- Patient Appointments.
- Patient Information.

Patient

- Patient Dashboard.
- Search Doctors.
- Book Appointment.
- My Appointments.
- AI Assistant.

---

4. Backend Technology

Python

Python will be the primary programming language for the backend.

Python is suitable for the project because it supports:

- Web development.
- Artificial Intelligence.
- Data processing.
- API development.
- Database integration.

FastAPI

FastAPI will be used to build the backend REST API.

The backend will handle:

- Authentication.
- User management.
- Doctor management.

- Patient management.
- Appointment management.
- Database communication.
- AI Agent tools.
- Notifications.

Example Backend Modules

The backend may be organized into the following modules:

- Authentication Module.
- Users Module.
- Doctors Module.
- Patients Module.
- Appointments Module.
- Departments Module.
- Notifications Module.
- AI Agent Module.

---

5. Database Technology

PostgreSQL

PostgreSQL will be used as the main relational database.

The database will store information about:

- Users.
- Doctors.
- Patients.
- Nurses.
- Departments.

- Appointments.
- Prescriptions.
- Notifications.

---

6. ORM

SQLAlchemy

SQLAlchemy will be used to connect the FastAPI backend with the PostgreSQL database.

It will allow developers to work with database tables using Python objects.

SQLAlchemy will help manage:

- Database models.
- Queries.
- Relationships.
- Database operations.

---

7. Data Validation

Pydantic

Pydantic will be used for data validation in the FastAPI backend.

It will validate incoming data.

Examples include:

- User registration data.
- Login data.
- Patient information.
- Doctor information.
- Appointment data.

Pydantic will help ensure that data follows the correct format before being processed.

---

8. Authentication

JWT Authentication

JSON Web Tokens (JWT) will be used for authentication.

The authentication process will include:

1. User login.
2. Username and password verification.
3. JWT token generation.
4. Token returned to the frontend.
5. Token sent with authorized requests.
6. Backend verification.

Role-Based Access Control

The system will support different user roles.

These roles include:

- Administrator.
- Doctor.
- Nurse.
- Receptionist.
- Patient.

Each role will have different permissions.

For example:

The Administrator can manage doctors.

The Doctor can access authorized patient information.

The Patient can access only their personal information and appointments.

---

9. AI Agent Technology

The AI Agent will be developed using Python.

The Agent will receive user requests written in natural language.

The Agent will:

1. Receive the user request.
2. Understand the request.
3. Identify the user's intention.
4. Determine the required action.
5. Select an appropriate tool.
6. Call the backend tool.
7. Process the result.
8. Return a response.

---

10. AI Agent Tools

The AI Agent will use controlled tools.

The initial tools include:

Search Doctor Tool

Searches for doctors using:

- Doctor name.
- Department.
- Specialization.

Check Availability Tool

Checks available appointment slots.

Book Appointment Tool

Creates an appointment after user confirmation and authorization.

Get Appointment Information Tool

Retrieves appointment information.

Get Patient Information Tool

Retrieves authorized patient information.

Generate Report Tool

Generates hospital statistics and reports.

---

11. AI Agent Safety

The AI Agent should not have unrestricted access to the database.

The recommended architecture is:

AI Agent

↓

Authorized Tool

↓

Backend API

↓

Database

The AI Agent should interact with hospital data through controlled backend functions.

The system should validate permissions before accessing sensitive information.

The AI Agent will not provide a final medical diagnosis.

---

12. API Architecture

The backend will use a REST API architecture.

Example API endpoints include:

Authentication

POST /auth/register

POST /auth/login

Doctors

GET /doctors

GET /doctors/{doctor_id}

POST /doctors

PUT /doctors/{doctor_id}

DELETE /doctors/{doctor_id}

Patients

GET /patients

GET /patients/{patient_id}

POST /patients

PUT /patients/{patient_id}

Appointments

GET /appointments

POST /appointments

PUT /appointments/{appointment_id}

DELETE /appointments/{appointment_id}

---

13. Frontend and Backend Communication

The system will follow this architecture:

Frontend

↓

REST API Request

↓

FastAPI Backend

↓

Business Logic

↓

Database

↓

Response

↓

Frontend

The frontend will communicate with the backend through API requests.

---

14. Development Environment

The development environment will include the following tools.

Visual Studio Code

Used for writing and managing the project code.

Python

Used for backend and AI Agent development.

Node.js

Required for React.js development.

PostgreSQL

Used for database management.

Git

Used for version control.

GitHub

Used for hosting the project repository and team collaboration.

---

15. Proposed Project Structure

The project may be organized as follows:

```
Hospital_AI_Agent
 ├── frontend
 │   ├── src
 │   ├── components
 │   ├── pages
 │   └── services
 │
 ├── backend
 │   ├── app
 │   │   ├── models
 │   │   ├── schemas
 │   │   ├── routers
 │   │   ├── services
 │   │   ├── database
```

```
│ │ └── main.py
│
├── ai_agent
│ ├── agent.py
│ ├── tools
│ └── prompts
│
├── database
│ └── migrations
│
├── docs
│
└── README.md
```

---

16. Development Workflow

The recommended development workflow is:

Step 1

Create the GitHub repository.

↓

Step 2

Create the project folders.

↓

Step 3

Set up the Python backend environment.

↓

Step 4

Set up PostgreSQL.

↓

Step 5

Create the database models.

↓

Step 6

Develop backend APIs.

↓

Step 7

Test the APIs.

↓

Step 8

Create the React frontend.

↓

Step 9

Connect the frontend with the backend.

↓

Step 10

Implement the AI Agent.

↓

Step 11

Connect the AI Agent with backend tools.

↓

Step 12

Test the complete application.

---

17. Required Software

The development team should install:

- Python.
- Visual Studio Code.
- Node.js.
- PostgreSQL.
- Git.

Optional tools:

- Postman for API testing.
- pgAdmin for database management.

---

18. Initial Python Packages

The backend may initially use:

- fastapi
- uvicorn
- sqlalchemy
- psycopg
- pydantic
- python-jose
- passlib

Additional AI-related packages will be selected when the AI Agent implementation begins.

---

19. Team Development Strategy

The team should divide the work into clear parts.

Possible responsibilities include:

Backend Developer

Responsible for:

- FastAPI.
- APIs.
- Database connection.
- Authentication.

Frontend Developer

Responsible for:

- React.js.
- User interfaces.
- Dashboard.
- API integration.

Database Developer

Responsible for:

- Database design.
- PostgreSQL setup.
- Tables.
- Relationships.

AI Developer

Responsible for:

- AI Agent.
- Agent tools.
- Natural language processing.
- AI integration.

---

20. Development Priority

The recommended order is:

1. Project setup.
2. Backend environment.
3. Database.
4. Authentication.
5. Doctor management.
6. Patient management.
7. Appointment management.
8. Frontend.
9. Dashboard.
10. AI Agent.
11. Notifications.
12. Advanced AI features.

---

21. Conclusion

The Hospital AI Agent will be developed as a modern full-stack web application.

React.js will provide the user interface, FastAPI will provide the backend APIs, PostgreSQL will store the system data, and Python will be used to develop the AI Agent.

The system architecture is designed to allow the project to grow gradually from a basic MVP into a more advanced AI-powered smart hospital management system.